

MCP

Model Context Protocol

my course notes · Claude Academy · Sept 2026



1. WHAT IS MCP?

- MCP is an **open standard** that lets AI apps connect to outside **tools, data and workflows** in one common way.
- Think of it like **USB-C for AI**: one standard plug, so any AI app can plug into any service.

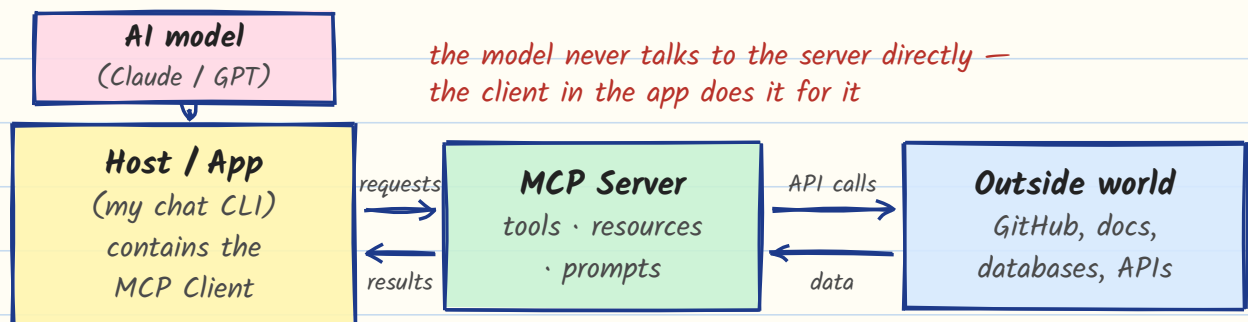
The problem it solves

Without MCP → you write and maintain ALL the tool code yourself (tool schemas, API calls, updates when the API changes). **With MCP** → someone builds a server **ONCE** and every app just connects to it.

2. WHY IT MATTERS

- Integrations are built once and **reused** everywhere.
- Works with **any model** → MCP doesn't care which AI sits on top (I ran my project on OpenAI!).
- Clean split: **app logic** stays in the app, **tool logic** stays in the server.

3. THE BIG PICTURE



4. KEY PLAYERS

- **Host / App** → the program the user actually uses (e.g. `main.py` chat).
- **MCP Client** → lives inside the app, talks to **one** server (1 client : 1 server).
- **MCP Server** ⇒ offers tools, resources and prompts.
- **Transport** → how client and server talk. `stdio` = local process (what I used), `HTTP` = remote server.

They talk in simple messages: "list your tools?" → list, "call this tool" → result.

5. THE 3 PRIMITIVES

★ EXAM FAV!

Primitive	Who controls it?	What for	In my project
TOOLS	the MODEL	actions: read,	<code>read_doc_content</code>
	(AI decides)	edit, do things	<code>edit_document</code>
RESOURCES	the APP	data put into	<code>@deposition.md</code>
		the context	
PROMPTS	the USER	ready-made	<code>/summarize</code>
		workflows	<code>/format</code>

Memory trick

Tools → **AI decides** · Resources → **App fetches** · Prompts → **User triggers**

- User clicks a button / types a `/command` → that's a **Prompt**.
- ! "Functions" is **NOT** an MCP primitive — only these 3!

6. BUILDING A SERVER (PYTHON, FASTMCP)

```
from pydantic import Field
from mcp.server.fastmcp import FastMCP
from mcp.server.fastmcp.prompts import base # needed for prompts!

mcp = FastMCP("DocumentMCP")
```

Tool (the AI calls it by itself)

```
@mcp.tool(name="read_doc_content", description="Read a document")
def read_doc(doc_id: str = Field(description="ID of the doc")):
    if doc_id not in docs:
        raise ValueError("Doc not found") # error goes back to the AI
    return docs[doc_id]
```

- The AI only sees the **name + description + Field descriptions** → write them clearly!

Resources (2 types)

```
# Direct (static URI) -> always the same thing
@mcp.resource("docs://documents", mime_type="application/json")

# Templated -> {doc_id} becomes a function argument
@mcp.resource("docs://documents/{doc_id}", mime_type="text/plain")
def fetch_doc(doc_id: str) -> str: ...
```

- One templated resource = **every** document. **mime_type** tells the client how to read it.

Prompt (user runs it as a /command)

```
@mcp.prompt(name="summarize", description="Summarize a doc")
def summarize(doc_id: str = Field(...)) -> list[base.Message]:
    return [base.UserMessage(f"Summarize {doc_id} ...")]
```

7. BUILDING THE CLIENT

The client uses a **session** to ask the server for things:

```
await session.list_tools()           # -> result.tools
await session.call_tool(name, args)  # -> tool result
await session.list_prompts()         # -> result.prompts
await session.get_prompt(name, args) # -> result.messages
await session.read_resource(AnyUrl(uri)) # -> result.contents[0]
```

- `read_resource`: check the **mimeType** → JSON gets `json.loads()`, text is returned as-is.

8. HOW A TOOL CALL FLOWS

- ① User asks "What's in report.pdf?"
- ② App sends the question + the tool list to the AI
- ③ AI replies "please call `read_doc_content(report.pdf)`"
- ④ Client runs `call_tool` on the MCP server
- ⑤ Result goes back to the AI
- ⑥ AI writes the final answer for the user

can repeat!

In the code this shows up as **stop_reason = "tool_use"** — the app keeps looping until the AI stops asking for tools.

9. TESTING WITH MCP INSPECTOR

- Run: `uv run mcp dev mcp_server.py`
- Open the **full URL** (with the token) → click **Connect**.
- Tools tab → Run Tool · Resources → List Templates · Prompts → Get Prompt
- Works in Inspector but not in the app? ⇒ bug is in the **client / chat code**, not the server.
- Edits reset when the server restarts (docs live in memory).

10. MISTAKES I MADE (DON'T REPEAT!)

- x Missing comma in `@mcp.tool(name=..., description=...)` → **SyntaxError**.
- x Forgot `from mcp.server.fastmcp.prompts import base` → **InvalidSignature** error.
- x Edited `.venv/.gitignore` instead of the **root** `.gitignore`.
- ! **NEVER** push `.env` (API key!) → always check git status before committing.

11. QUIZ RECAP

- ? Q: Without MCP, the main problem?
writing and maintaining ALL the tool code yourself
- ? Q: User clicks a "summarize" button?
PROMPTS (user controls when it starts)
- ? Q: `docs://documents/report.pdf`?
a **TEMPLATED** resource (`{doc_id}` in the URI)
- ? Q: The 3 primitives?
Tools, Resources, Prompts

ONE-LINE SUMMARY



MCP = a standard way to plug tools, data and prompts into ANY AI app.